Vision for Autonomous Robots

Lab Project 2: Detection & Localisation

Lisann Becker (12558141)
Batu Helvacioğlu (14992760)
Marina Orozco González (15214052)

0 Introduction

Localization is an essential element of robot control. The ability to autonomously find its way around an environment where GPS signal is scarce or non-existent, is key for the robot's capabilities in carrying out its directives. In this lab we drive in a closed environment, employing reference points which the robot can use to triangulate its position in the real space.

The reference points we leverage for this approach were Aruco markers strategically placed around the lab's soccer field to provide positional data. By using the camera, the robot can detect these markers and compute its relative position with respect to them.

In this report, we outline the steps we took to complete the task of a curling match where a RAE robot had to identify AruCo markers spread around the vicinity of the playing field, triangulate its position relative to the markers, and then move to a target location within the field while simultaneously avoiding obstacles.

1 Section 1: Marker detection by a RAE robot

1.1 Code implementation

In order to detect Aruco markers in the images, we utilized cv2.aruco library. Images were undistorted and converted to gray-scale as preprocessing steps. Additionally we increased the resolution of the captured images from 640x400 to 1280x800 by changing the camera.yml config file of RAE robot.

1.2 Robustness Test

We collected around 20 images from different locations around the pitch. From these images, Figure 1 highlights the top six images that best represents different angles, distances and edge cases.



Figure 1: Set of images from a variety of locations and distances to test robustness.

Images 1 and 4 were both taken from the center of the pitch, facing the window, but they feature different image calibrations. These differences can be observed in the small variations in the bottom-left corners of the images. We included both to test our method's robustness against varying image calibrations.

In Image 1, our method detected only the leftmost marker, whereas in Image 4, it successfully identified the middle and rightmost markers. In both cases, the small markers on the columns and the medium markers on the windows were not detected. It is worth noting that in Image 1, the rightmost marker was occluded, so its detection was not expected anyways.

Image 2 is located closer to the windows than Images 1 and 4, and the markers present different sizes. We can see how the algorithm keeps struggling with detecting the marker on the middle, even if it was closer to it.

Images 3 and 5 include the markers around the two goals. While Image 3 is from a short distance (penalty spot) and markers were detected, we can see how in Image 5, which is from a longer distance (outside the penalty box) and a not so perpendicular angle, the algorithm struggles with detecting the markers. This is even more obvious in Image 6, where no marker is detected.

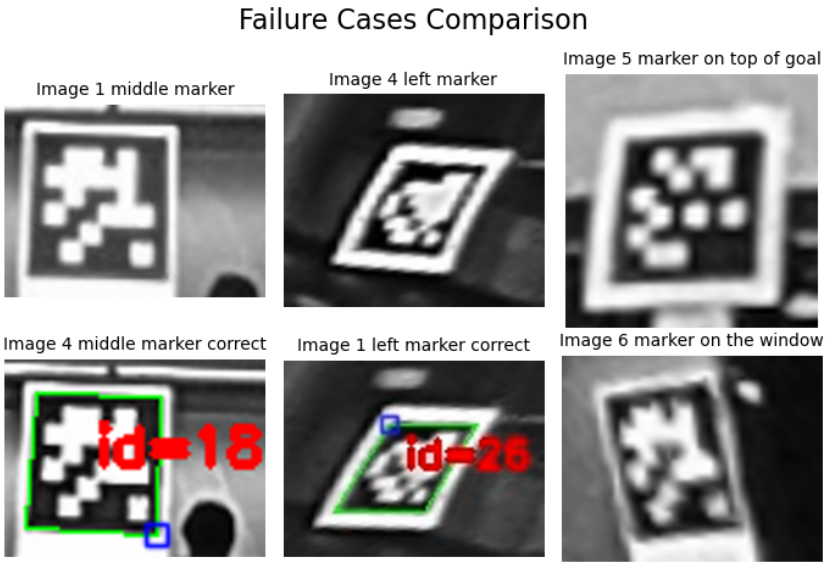


Figure 2: Zoomed in images of markers that were not detected in the robustness test.

1.3 Failure Cases

We analyzed the failure cases by closely inspecting the markers that were not detected. The failure cases were under different circumstances such as different angle, distance, marker size and image distortion. However, the root cause of the failures seemed to be the same: As it can be seen in Figure 2, the markers are blurry and the white parts of the markers seem to blend in with the black parts.

If we compare the middle and left markers from Image 1 and Image 4 (with a different camera calibration), we can see that the black regions of the marker in Image 4 are more visible and the edges of the white regions are sharper. Black regions in Image 1 are slightly larger. Same happens in Images 5 and 6, where we can again observe that the markers are blurry and the black regions are not clearly defined.

Even though the markers in these failure cases varying sizes and are at different distances and angles, or the images are with different calibration, we believe that the common cause for failure is lack of sharpness. This is more obvious in markers that are farther away from the center of the image. The least blurry marker is the Image 1 middle marker, which is the most centered one. We were able to detect this marker in the distorted version of the image but not on the undistorted version. This again supports our hypothesis that the markers are not sharp enough, since undistorting slightly reduces the sharpness.

1.4 Improving Robustness

We applied the `unsharpening` technique, which enhances the image by blurring it and then subtracting the blurred version from the original. To introduce the blur, we used `cv2.GaussianBlur` with a 3x3 kernel and a σ value of 5. Then, we sharpened the image using `cv2.addWeighted` with $weight = 3$ for original image and -2 for blurred image and $gamma = 0$.

With the default parameters above, we observed an overall performance improvement. However, some markers that were previously detected were no longer identified. To address this, we conducted a grid search using the number of detected markers as the performance metric to optimize the unsharpening parameters. The optimal settings were a Gaussian blur with a kernel size of (3, 3) and $\sigma = 2$, along with a weighted combination where the original image weight was 1, the blurred image weight was -0.5, and γ was set to 0.

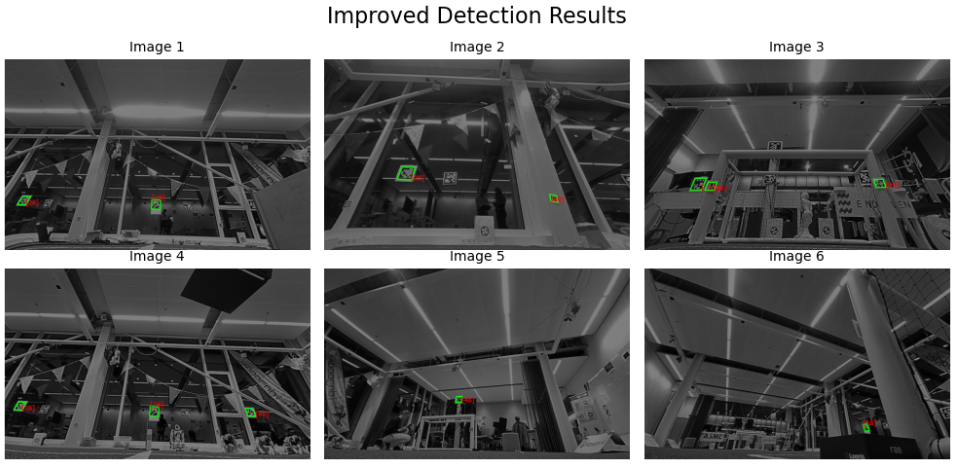


Figure 3: Improved detection results on the robustness test images.

1.5 Comparison and Discussion

We managed to increase the number of detected markers from 38 to 45 in our 22 test images. More importantly, we were able to detect at-least 1 marker in all of the images with our final solution even in the edge case in Image 6. Our improved detection results on the robustness test images can be seen in Figure 3.

2 Section 2: Localization by a RAE robot

2.1 Report the Distance Measurement

To measure the distance between the robot and the marker, we use the function `cv2.aruco.estimatePoseSingleMarkers` to compute the robot's relative distance to each marker on each dimension x, y and z. This function is currently deprecated according to the documentation, but it is however the function that worked best for us.

The results of the computed relative distance to the markers in images A, B and C from Figure 4 can be seen in Figure 5, or in more detail in Table 1 from the Appendix. Note that the coordinates are measured from the center of the field, with the positive Y coordinate point at the windows of the lab.

2.2 Reduce the error of distance measurement

We can see that the previous function gives us really wrong values of the relative coordinates, specially concerning the Y and Z coordinates of the robot respect to the markers. To improve this, we apply the nuclear norm from numpy, `np.linalg.norm`, over the vector we get as output from aruco, to find the perpendicular distance between the robot and each marker. The results from the previously analyzed images can be checked in Figure 6, or in more detail in Table 2 in the appendix.



Figure 4: From left to right: Images A, B, C used for Section 2.

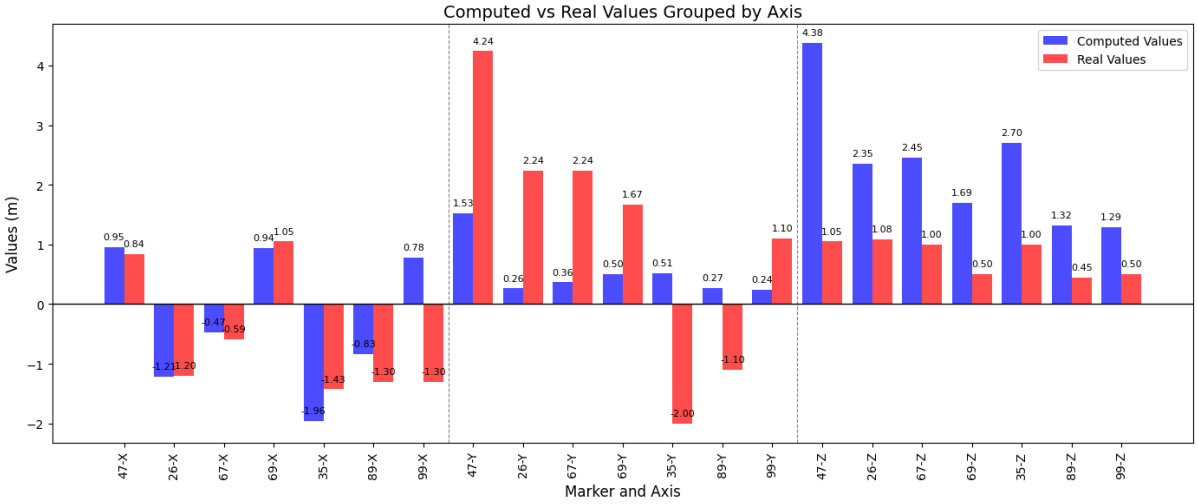


Figure 5: Mean relative distance computed on each coordinates relative to the real ones for the detected markers in the images. Image A: Marker 47; Image B: Markers 26, 67, 69; Image C: Markers 35, 89, 99.

We can check that these computed distances are very close to real distances, specially in contrast with the estimated vectors from aruco function that didn't performed as well. This takes us to the conclusion that the module of the vector given by aruco is reliable even if the coordinates themselves aren't accurate.

2.3 Do triangulation and report results

In order to find the real-world space coordinates of the RAE robot making use of the distances from the robot to the marker as computed in the previous subsection, we will also make use of the known coordinates of the markers in the 3D world making use of **triangulation**.

With it, we look for the intersection of the spheres that are created around each marker's center, with their distance to the robot as radius, taking into account that the robot is always going to be on the floor: this is, the spheres will intersect with the $z_{global} = 0$ plane, forming two circles. This idea is represented in Figure 7.

Mathematically, this translates to finding the point (X_r, Y_r, Z_r) which satisfies Equation 1.

$$\begin{cases} (X_r - X_{m1})^2 + (Y_r - Y_{m1})^2 + (Z_r - Z_{m1})^2 = d_{r1}^2 \\ (X_r - X_{m2})^2 + (Y_r - Y_{m2})^2 + (Z_r - Z_{m2})^2 = d_{r2}^2 \\ Z_r = 0 \end{cases} \tag{1}$$

Where $(X_{m1}, Y_{m1}, Z_{m1}), d_{m1}$ and $(X_{m2}, Y_{m2}, Z_{m2}, d_{m2})$ are the coordinates and distances to the robot of markers 1 and 2, respectively.

Given the quadratic nature of the expressions, we will always end up with two possible solutions. However, most of the times only one of them will be within the margins of the soccer field. When both of them lie within the soccer field, we juts randomly select one, but this case shouldn't appear if the distances are computed correctly.

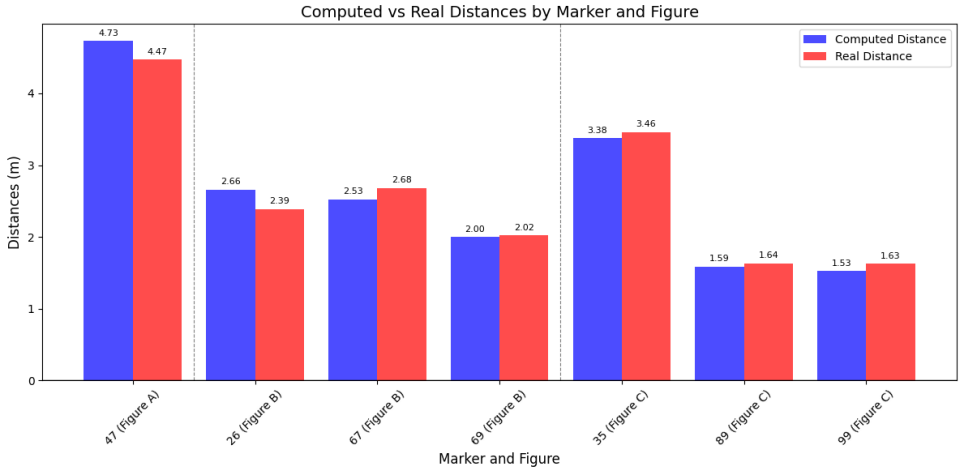


Figure 6: Computed and real distances for the different markers.

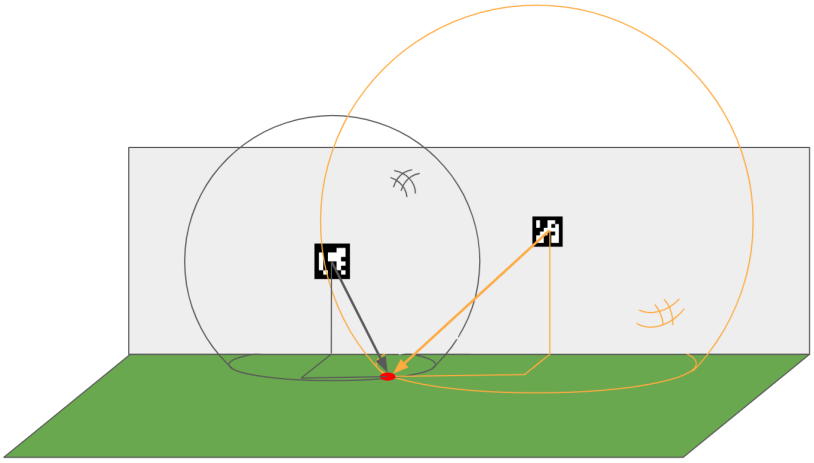


Figure 7: 3D representation of intersection among the spheres formed by the estimated distance between the markers and the robots and the floor.

There are some scenarios where this function might fail and will not provide any results. This would be the case if the estimated distances aren't accurate enough and the spheres don't intersect in $Z = 0$.

We can use two markers of Figures 2 and 3 (as Figure 1 doesn't contain more than one identified marker) to try the triangulation approach. The results are shown in Figure 8, where we can see a picture of the view of the robot on the left, with the detected marker highlighted in green and with their corresponding ID next to them. On the right, a bird view of the field with all the markers and the global coordinates of the robot in a red spot. The ducks are represented with yellow dots. The black points will be explained later

2.4 Update your algorithm, and do triangulation on at least three reference points

As Expression 1 is valid for a couple of markers, when we have more than two, we compute it for all possible pairs of markers, and then we take the average over the resulting coordinates. This should improve the performance of the detections the error will be averaged over the different combinations.

In Figure 9 we can see our localization results when using three markers in the picture.

We can see that the results for the image above are the same as when we only used two points. This happens here because there weren't intersecting circles among the other markers in the image, and thus they weren't taking into account. Even though the performance didn't thus improve, it demonstrates

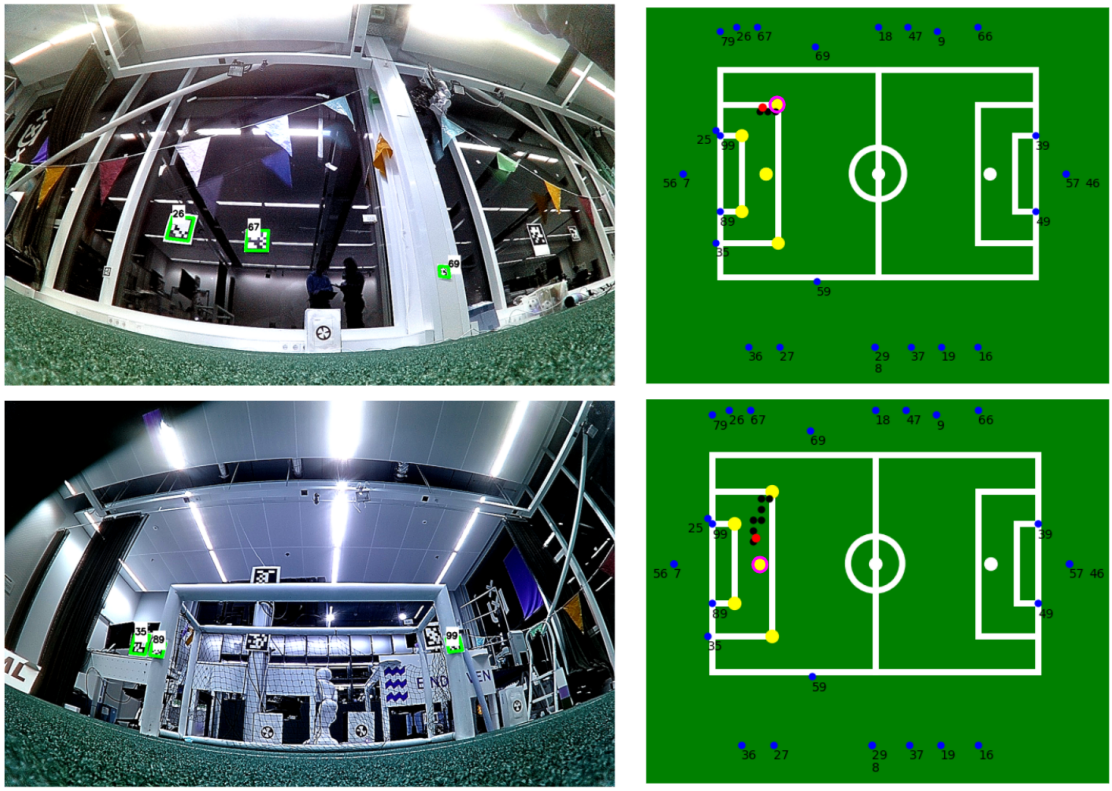


Figure 8: **Image above**: Robot's coordinates for Image B. Real coordinates (pink circle): $(-2.85, 2.00, 0)$m, Computed coordinates (red dot): $(-3.307, 1.923, 0)$m. Markers 26 and 67 were used. **Image below**: Robot's coordinates for Image C. Real coordinates (pink circle): $(-3.200, 0, 0)$m, Computed coordinates (red dot): $(-3.294, 0.710, 0)$m. Markers 89 and 99 were used.

that our algorithm is also robust when less intersections are found. In the second figure we see how the localization actually improves.

2.5 Real-time Localize the RAE robot during its movement

Real-time localization during movement is more challenging due to the motion blur present in the captured images as the camera moves, increasing the difficulty of correctly identifying the markers.

To add more robustness to the localization, we also implemented a way of computing the localization with only one marker. For this, we take the x-coordinate from the `cv2.aruco.estimatePoseSingleMarkers` function as true, as this lateral displacement from the marker is more robust than the depth one. Knowing that the robot is on $Z = 0$, given the distance to the marker d_m, the problem becomes the search of the intersection among the sphere of radius d_m around the marker, and the line of $X = X_0$ (or $Y = Y_0$ if we are not looking at the windows or the wall), being X_0 or Y_0 the lateral distance that came out from the aruco function. We can see an example of localization with only one marker in Figure 10.

When localizing in real-time, we cannot reliably compare the results with the real coordinates, but when the robot moves relatively slowly, the localization seems to be working. We can see one more example of it in Figure 11.

3 Section 3: Using RAE robot for curling match

3.1 Driving to the target

We implemented a planning algorithm using A* search to find the shortest path to the goal location. For the purposes of the curling match, we adapted the pseudocode from the above link with the following inclusions:

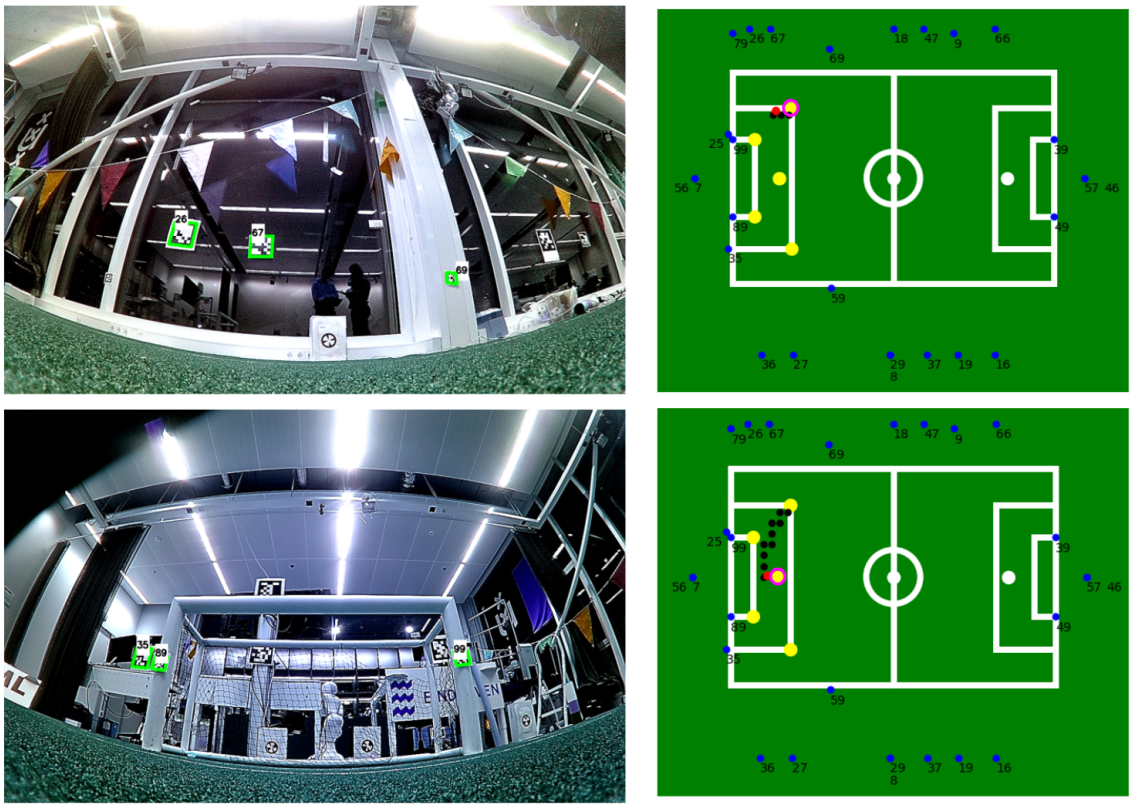


Figure 9: Robot localization making use of the three detected markers. **Image above**: Robot's coordinates for Image B. Real coordinates (pink circle): $(-2.850, 2.000, 0)$m, Computed coordinates (red dot): $(-3.307, 1.923, 0)$m. Markers 26 and 67 were used for computing this result. **Image below**: Robot's coordinates for Image C. Real coordinates (pink circle): $(-3.200, 0, 0)$m, Computed coordinates (red dot): $(-3.506, 0.052, 0)$m. Markers 89 and 99 were used for computing this result.

1. The real-world coordinates of the field are translated into graph coordinates where each point corresponds to a specific quadrant in the real-world field.
2. Other targets (e.g. goal states), as well as any other detected RAE robots are considered as obstacles and therefore invalid points for the robot to go through. In order to guarantee avoidance, we can also consider coordinates in the immediate vicinity of the obstacles.
3. The required heuristic function for A* is either the shortest Manhattan distance (also known as Taxicab distance), or Euclidean distance, both in graph-coordinate space. We find Manhattan distance to perform more robustly in our use-case, thus we employ it throughout our tests.

In the previous Figures 8, 9, 10 and 11 we can see with black dots the paths that have been computed from each localization to the goal-duck.

A general outline for our proposed control is outlined in Algorithm 2. After the optimal path is found, the robot will move to the appointed coordinates by translating the coordinates into movement directives, such as `MOVELEFT`, `MOVERIGHT`, `MOVEFORWARD`.

3.2 Detect other RAE robots and bypass them

Our approach if based on the idea that when a robot is in front of us a large amount of black pixels will be shown in the lower seventh of the input frame, reducing the noise that background objects might cause. Using proportion of blackness proved as a viable strategy to detect a robot in our lab's environment, as the robot is the only object that will appear black. The mechanism consists of an ongoing running robot detection part that triggered a second robot dodging algorithm whenever consistently detecting one or multiple robots in the input frames.

Empirical testing showed that the brightness threshold of 48 and black proportion of 18% of the mask

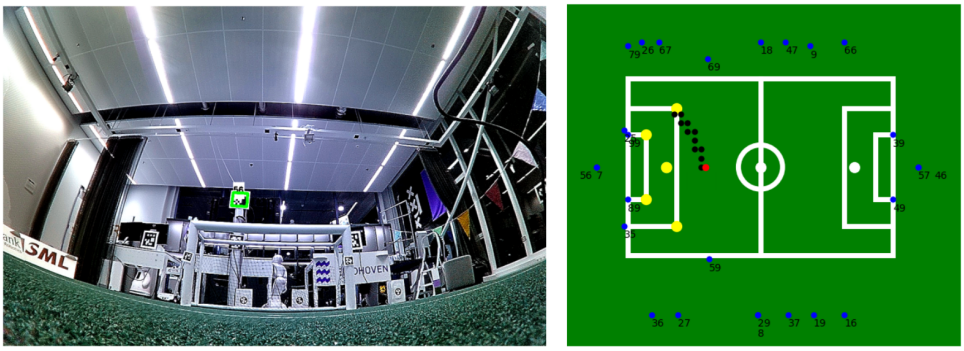


Figure 10: Robot localization making use of only one marker. The robot's coordinates for this case are $(-1.88, 0, 0)$m.

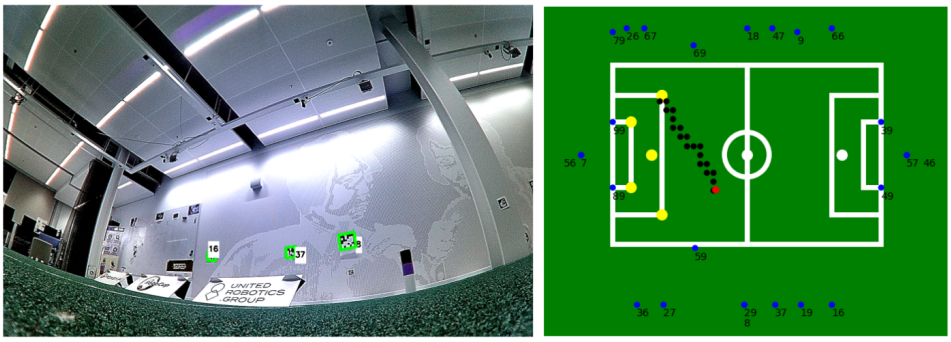
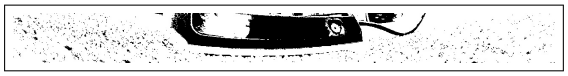


Figure 11: Example of real-time robot localization during movement.

were reliable parameters to detect a robot in the lab environment and from adequate distance 12. Moreover, the detection algorithm robustly detected multiple robots, and from even greater distances. For more details, the robot detection algorithm is summarized in the Appendix, in Algorithm 3.



(a) Mask Image



(b) Annotated Original

Figure 12: Visualization of the robot detection process: (a) the binary mask of the detection region and (b) the original annotated image.

Dodging was initiated whenever the we consistently detected robots across a history of 10 frames, which is equivalent to one second of camera input. The dodging maneuver itself included a left turn which ended as the obstacle left the field of view. We chose a left over right turn, as the robot relied on its right camera input, allowing it to maintain a better perspective of the obstacle during the maneuver. The dodging algorithm is detailed in Algorithm 1.

While focusing on proportion of blackness limited our algorithms to dodging dark objects instead of robots specifically, this proved as highly effective strategy in the lab environment. Utilizing detection history made the algorithm robust, to the point where it never initiated dodging unnecessarily, while always initiating dodging when a robot did occur in the image.

Algorithm 1 Obstacle Dodging Based on Robot Detection History

Require: Robot detection history $R_{\text{detected}}[t-10:t]$
Ensure: Perform dodge maneuver if robot is detected persistently over 10 frames
if $\sum R_{\text{detected}}[t-10:t] = 10$ **then** ▷ Check detection over the last 10 frames
  Initialize: Twist message T
  T.linear.x $\leftarrow 0.1$ ▷ Set forward speed
  T.angular.z $\leftarrow 0.6$ ▷ Set left turn rate
  Publish T and execute for 0.5 seconds
  T.angular.z $\leftarrow -0.6$ ▷ Adjust to turn right slightly
  Publish T and execute for 0.2 seconds
end if

3.3 Run your RAE robot for curling match

Localization performance. Localization depended on marker detection (Section 1) and triangulation (Section 2.3) and varied significantly between time steps.

While the robot was able to detect markers most of the time, it relatively frequently misclassified them or hallucinated (see Figure 13), leading to variance in estimated distances and deviations in the position estimates. Downstream, this inconsistency in marker detection could prompt wrong movement commands, despite triangulation and navigation functioning as expected.

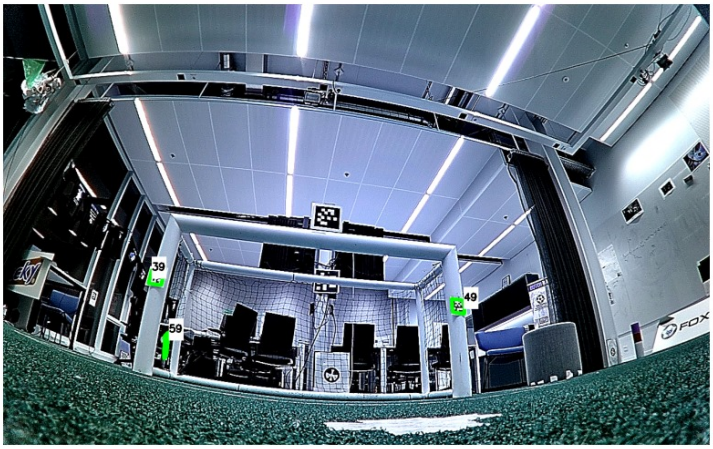


Figure 13: Example for marker hallucination - the detection of marker 59 posed a false positive. Hallucination and variability in marker detection were major challenges to our localization performance.

When multiple markers were detected, triangulation proved successfully by averaging over marker pairs. While the performance dropped when only visualizing a single marker, as the robot kept moving, the robot localization corrected itself when multiple markers were in view again.

Movement behavior. The robot's movement was generated from the A* path planning in Algorithm 2 (Appendix), and generally performed as expected. When the robot had sufficient information, the translation of graph coordinates into movement commands ("left", "right" and "forward") worked effectively. However, the robot often over-adjusted to the right while navigating and despite sending left-bound movement signals and proportionally strong left movement commands. This sometimes resulted in the robot getting stuck in loops, particularly when markers were not present.

Dodging performance. The obstacle detection and dodging mechanism worked reliably. Triggering the dodge maneuver (Algorithm 1) only after detecting robots across 10 consecutive frames ensured robustness by preventing false-positive reactions while not overly constraining the algorithm's onset.

However, change of orientation due to dodging could lead to markers disappearing from the robot's field of view, leading to the navigation issues described above.

Overall, the described pipeline successfully integrated localization, navigation and dodging, enabling the robot to complete curling tasks. Improving marker detection accuracy and addressing movement issues would further enhance overall performance.

3.4 Discussion & Analysis

Our first recurring issue was variability in marker detection. Specifically, while the detection system worked reliable in offline tests, it struggled during live operation. This caused localization variability and led to incorrect paths being calculated. The variability might stem from differences in the live video stream, such as motion blur or lighting changes. Moreover, we currently include multiple marker families (e.g., AruCo and AprilTag), which may have increased hallucinations and faulty detections.
Incorporating a history-based detection could stabilize detection and downstream localization. Specifically, we could save a history of detected markers over the past frames and select only the most consistently observed markers for localization, reducing variance and false positives. This strategy is viable as, despite several faulty detections, the correct marker types are detected most commonly.

Secondly, the robot sometimes prematurely concluded it had reached its goal location, likely due to main issue of localization noise described above: the robot might incorrectly conclude to have reached its goal location when one of the varied location estimates indicates so.
To improve this, we propose adding a stopping verification step. After the robot first determines it is near its goal, it should continue localization for a few more frames while halting or moving slowly, to confirm its position. Only when multiple consistent localization support that the goal has been reached should the robot end its run.

During the demo, our algorithm exceeded expectations, successfully navigating the robot to the duck despite encountering control issues when turning left, presumably due to hardware limitations. We believe the success of the algorithm is largely attributed to the constant localization updates, which we were able to compute even when only a single marker was visible. This allowed the algorithm to correct its path in subsequent frames, even if a marker was incorrectly detected in one frame.

4 Conclusion

In this lab we have gotten familiar with aruco markers, which came with several functions to identify and measure the relative distance to them. While these functions were easy to implement, their results weren't super accurate with the resolution we are using in RAE robots, leading to wrong or none identification of the markers in various situations.

For the cases where they worked, we were able to localize the robot in real world coordinates, as the the coordinates for the markers were known. Triangulation seemed to work the best, specially when dividing the error with several pairs of markers. However, we also implemented a way of localizing when only detecting one marker.

Making use of this localization and of the A* search algorithm, we planed the path that the robot should follow from any localization to arrive to its goal-duck. The control instructions were given through a path-coordinates to `MOVELEFT`, `MOVERIGHT`, `MOVEFORWARD` orders. Moreover, a robot avoidance algorithm was also implemented, even though we didn't run it in the final test of playing a curling match, to reduce the possible sources of errors by making the problem easier.

Overall, we can conclude that we succesfully solved the different challenges of this lab, while getting knowledge of all the limitations and difficulties that can arrise from a task like this one in a limited hardware.

5 Appendix

A Relative distance robot-marker

	Marker ID	Axis	Computed Value (m)	Real value (m)
Image A	47	X	0.949	0.840
		Y	1.528	4.240
		Z	4.375	1.050
Image B	26	X	-1.212	-1.200
		Y	0.262	2.240
		Z	2.354	1.080
	67	X	-0.470	-0.590
		Y	0.365	2.240
		Z	2.454	1.000
	69	X	0.944	1.050
		Y	0.497	1.670
		Z	1.694	0.500
Image C	35	X	-1.960	-1.428
		Y	0.512	-2.000
		Z	2.702	1.000
	89	X	-0.834	-1.300
		Y	0.271	-1.100
		Z	1.322	0.450
	99	X	0.775	-1.300
		Y	0.238	1.100
		Z	1.295	0.500

Table 1: Results for the measurement of the relative distance robot-marker on different markers on different images with the first method

B Distances robot-marker

	Marker ID	Computed Distance (m)	Real Distance (m)
Figure 1	47	4.730	4.472
Figure 2	26	2.661	2.388
	67	2.526	2.682
	69	2.002	2.020
Figure 3	35	3.377	3.460
	89	1.586	1.635
	99	1.526	1.630

Table 2: Results for the measurement of the relative absolute distance robot-marker on different markers on different images with aruco function + module computation of the vectors. The real distances has been measured with the Bosch Zamo laser measure set.

C Adapted A* Search for Curling Match

Algorithm 2 Adapted A* Search for Curling Match

Require: Start position S, goal position G, graph representation of the field F, obstacle set O
Ensure: Path P from S to G, or failure if no path exists

Initialize:
Create an open set OPEN, initially containing only S.
Create an empty closed set CLOSED.
Assign $g(S) \leftarrow 0$ ▷ Cost from start to S
Compute $h(S, G)$ ▷ Manhattan distance from S to G
Set $f(S) \leftarrow g(S) + h(S, G)$
while OPEN is not empty **do**
    Extract the node N in OPEN with the lowest f-value
    if $N = G$ **then**
        Return the reconstructed path P from S to G
    end if
    Move N from OPEN to CLOSED
    for each neighbor n of N in F **do**
        if $n \in O$ or $n \in$ CLOSED **then**
            Continue to the next neighbor
        end if
        Compute tentative cost: $g_{\text{tentative}} \leftarrow g(N) + \text{cost}(N, n)$
        if $n \notin$ OPEN **or** $g_{\text{tentative}} < g(n)$ **then**
            Update $g(n) \leftarrow g_{\text{tentative}}$
            Compute $h(n, G)$ ▷ Manhattan distance from n to G
            Set $f(n) \leftarrow g(n) + h(n, G)$
            Add or update n in OPEN with the new $f(n)$
        end if
    end for
end while
Return Failure ▷ No path exists if OPEN is empty and G is not reached

D Robot detection

Algorithm 3 Detect Robot Based on Proportion of Black Pixels

Require: Image I
Ensure: Robot detected R_{detected}

$I_{\text{lower}} \leftarrow I[\frac{6H}{7} : H, :]$ ▷ Extract lower seventh of image
$G \leftarrow \text{Grayscale}(I_{\text{lower}})$
$B \leftarrow \text{Brightness}(G, \alpha = 1.5, \beta = 0)$ ▷ Enhance brightness
$M \leftarrow \text{Threshold Binary}(B, 48, 255)$ ▷ Binarization threshold 48
$\rho \leftarrow 1 - \frac{\text{Count Non-Zero, White Pixels}(M)}{\text{Total number of pixels}(M)}$ ▷ Compute black pixel proportion
$R_{\text{detected}} \leftarrow (\rho > 0.18)$ ▷ Detect robot
return R_{detected}